

Data Engineering Interview Prep

Delta Lake • SCD • Data Governance • AWS • Spark • Orchestration

Compiled Q&A Reference Guide

Q1. What is a Delta Table? How do you handle schema evolution and Slowly Changing Dimensions (SCD)? How do you recover accidentally deleted data?

What is a Delta Table

A Delta table is a table format built on top of Parquet files that adds a transaction log (`_delta_log`) on top of raw data files. This gives you:

- **ACID transactions** on data lakes
- **Time travel** (query previous versions)
- **Schema enforcement + evolution**
- **Upserts/deletes** (MERGE, UPDATE, DELETE) which plain Parquet doesn't support well
- **Unified batch + streaming** source/sink

Schema Handling

Schema enforcement: by default, Delta rejects writes that don't match the existing schema (prevents bad data from corrupting the table).

Schema evolution: when you do want to allow it, use the `mergeSchema` option:

```
df.write.format("delta").mode("append")
  .option("mergeSchema", "true").save(path)
```

For structural changes (rename, drop, type change), use `ALTER TABLE` or, for a full schema replace, the `overwriteSchema` option.

SCD (Slowly Changing Dimension) Approach

Type	Behavior	When to Use
SCD Type 1	Overwrite old value, no history kept	When historical values don't matter
SCD Type 2	Keep full history with effective_date / end_date / is_current flag	Most common — full auditability needed
SCD Type 3	Keep only the previous value in a separate column	Rare — limited history needs

Most enterprise pipelines default to **Type 2**. Typical implementation using `MERGE INTO` :

```
MERGE INTO target t
USING updates u
ON t.id = u.id AND t.is_current = true
WHEN MATCHED AND t.value <> u.value THEN
  UPDATE SET t.end_date = current_date(), t.is_current = false
WHEN NOT MATCHED THEN
  INSERT (id, value, start_date, end_date, is_current)
VALUES (u.id, u.value, current_date(), null, true)
```

This is typically done in two passes: first close out changed records, then insert the new current version.

Recovering Accidentally Deleted Data

This is where Delta's transaction log is a lifesaver:

- **Time Travel restore:** `RESTORE TABLE my_table TO VERSION AS OF 12` or `TIMESTAMP AS OF '2026-07-01'`
- **Query old version without restoring:** `SELECT * FROM my_table VERSION AS OF 12`
- This works because Delta doesn't immediately delete old Parquet files — it marks them removed in the log. They persist until `VACUUM` runs (default retention 7 days).

Best practice: Don't rely only on time travel — also do periodic snapshot exports (e.g., to a separate S3 path or Glacier), set `VACUUM` retention appropriately, and limit auto-vacuum on critical tables to preserve a real recovery window.

Q2. Difference between Snowflake Schema and Star Schema — when and why to use each?

	Star Schema	Snowflake Schema
Structure	Fact table + denormalized dimension tables	Fact table + normalized (multi-level) dimension tables
Joins	Fewer, simpler joins	More joins (dimensions split into sub-dimensions)
Query performance	Faster (fewer joins)	Slower (more joins), but less storage used
Redundancy	More data duplication	Less duplication, cleaner normalization
Maintenance	Easier for BI/reporting	More complex ETL to maintain normalization

When to use Star Schema: when read/query performance matters most — BI dashboards, data warehouses like Redshift/Snowflake/BigQuery. This is the most common choice today.

When to use Snowflake Schema: when storage cost or strict normalization matters more than query speed, or when dimensions have deep hierarchies (e.g., Product → Subcategory → Category → Department) reused across many facts that need a single source of truth.

In practice, most modern warehouses favor star schema because storage is cheap and compute/query speed matters more.

Q3. How do we secure PII data?

A layered approach is used:

- **Encryption at rest** (S3 SSE-KMS, Delta table encryption) and **in transit** (TLS)
- **Column-level masking/tokenization** — mask or hash PII columns (e.g., Unity Catalog column masking, dynamic views)
- **Row/column-level access control** — Unity Catalog, Ranger, or Lake Formation policies restricting who sees what
- **Data classification/tagging** — tag PII columns so governance tools can enforce policy automatically
- **Least privilege IAM** — separate roles for the raw PII zone vs the curated/anonymized zone
- **Audit logging** — track every access to PII tables
- **Anonymization/pseudonymization** in lower environments (dev/test) so PII never leaves prod unmasked

Q4. Why are UDFs not reliable/performant in Spark?

- **No Catalyst optimization:** Spark's SQL engine (Catalyst) can't see inside a Python/Scala UDF — it treats it as a black box, so it can't push down predicates, reorder, or optimize around it.
- **Serialization overhead** (Python UDFs especially): data has to move from the JVM to a Python process row-by-row (or via Arrow in batches for Pandas UDFs), which is expensive.
- **No vectorization** (regular UDFs): row-at-a-time processing instead of columnar/vectorized execution.

Mitigation:

- Prefer built-in Spark SQL functions whenever possible
- Use **Pandas UDFs (vectorized UDFs)** instead of regular Python UDFs — they use Arrow and process in batches
- Avoid UDFs entirely for simple transformations; only use them for genuinely custom logic

Q5. Difference between Amazon S3 and Amazon Glacier (storage cost/tiering)

	S3 Standard	Glacier
Purpose	Frequently accessed data	Long-term archival, rarely accessed
Retrieval time	Milliseconds	Minutes (Expedited) to hours (Standard/Bulk)
Storage cost	Higher	Much cheaper (fraction of S3 cost)
Retrieval cost	Low/free	Higher, especially for expedited retrieval
Use case	Active pipelines, dashboards, hot data	Compliance archives, backups, old logs

Intermediate tiers: S3 Standard-IA, S3 One Zone-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, Glacier Deep Archive (cheapest, 12+ hour retrieval).

Use **S3 Lifecycle policies** to auto-transition data (e.g., Standard → IA after 30 days → Glacier after 90 days → Deep Archive after 1 year).

Q6. How do you set up DAG-level dependency (not just task-level dependency) in orchestration tools like Airflow?

- **Task dependency:** Task B waits for Task A within the same DAG (e.g., `task_a >> task_b`).
- **DAG dependency:** An entire DAG waits on another DAG's completion — cross-pipeline orchestration.

How to implement DAG-level dependency in Airflow:

- **ExternalTaskSensor:** DAG B has a sensor task that polls until a specific task in DAG A completes.
- **TriggerDagRunOperator:** DAG A, at its end, triggers DAG B directly (push-based, more reliable than polling).
- **Datasets / data-aware scheduling** (Airflow 2.4+): DAG B is scheduled to run automatically when a Dataset that DAG A produces is updated — the modern, preferred approach, since it decouples DAGs from rigid timing and ties them to actual data availability instead.

Q7. With ~5 TB of data, how do you secure it and give row-level access to multiple different groups?

- **Storage security:** Encrypt at rest (KMS) and in transit; restrict bucket/table access via IAM roles per team/group.
- **Row-level security (RLS):**
 - Databricks: **Unity Catalog row filters** — a function filters rows based on the querying user's group (e.g., `current_user()` or `is_member('group_name')`), attached to the table.
 - Snowflake: **Row Access Policies**.
 - Redshift: RLS via views + `SESSION_USER` groups, or Lake Formation for Redshift Spectrum/Athena.
- **Column-level security:** dynamic masking for sensitive columns per group.
- **Group-based access:** map AD/IAM groups to data access policies — e.g., Group A sees only rows tagged `region='US'`, Group B sees `region='EU'`, off a single shared table rather than duplicating data.
- **Partitioning strategy** at 5TB scale: partition by the same key used for access control (e.g., `region`, `tenant_id`) so RLS also benefits from partition pruning for performance.
- **Audit + monitoring:** log all query access for compliance.

Q8. What is AWS Lambda?

AWS Lambda is a **serverless compute service** — you upload code (Python, Java, Node, etc.) and AWS runs it in response to triggers (S3 upload, API Gateway call, EventBridge schedule, Kinesis stream, SQS message) without you managing any servers. You pay only for execution time (millisecond-level billing).

Common data engineering use: triggering a transformation when a file lands in S3, lightweight ETL, orchestration glue, or invoking a Glue job/Step Function.

Q9. For masked data, how can one group see real values while another cannot — how is that permission set up?

Core Idea

You don't create two copies of the table. You attach a **masking function** to the column that checks the querying user's identity/group at query time, and returns either the real value or a masked one.

Databricks (Unity Catalog) — most common in the Spark world

```
CREATE FUNCTION mask_pii(email STRING)
RETURNS STRING
RETURN
CASE
  WHEN is_member('pii_full_access') THEN email
  ELSE CONCAT('***MASKED***', SUBSTRING(email, -4))
END;

ALTER TABLE customers ALTER COLUMN email
SET MASK mask_pii;
```

`is_member()` checks if the current user belongs to a group/role. Whoever queries `SELECT email FROM customers` gets the real or masked value automatically — no separate table or view needed.

Snowflake — Masking Policies

```
CREATE MASKING POLICY email_mask AS (val STRING) RETURNS STRING ->
CASE
  WHEN CURRENT_ROLE() IN ('PII_ADMIN') THEN val
  ELSE '***MASKED***'
END;

ALTER TABLE customers MODIFY COLUMN email
SET MASKING POLICY email_mask;
```

Same concept — role-based, applied at the column level, enforced automatically for every query.

Redshift — Dynamic Data Masking

```
CREATE MASKING POLICY mask_email
WITH (email VARCHAR(256))
USING (CASE WHEN current_role = 'analyst' THEN '***' ELSE email END);

ATTACH MASKING POLICY mask_email ON customers(email) TO ROLE analyst;
```

If the Platform Doesn't Support Native Masking

Use views instead:

```
CREATE VIEW customers_masked AS
SELECT
  id,
  CASE WHEN current_user() IN (SELECT user FROM pii_access_group)
    THEN email
    ELSE 'MASKED'
  END AS email
FROM customers;
```

Grant access to the **view**, not the base table — analysts only ever query the view, while admins can query the base table directly.

Row-Level + Column-Level Together

- **Row filter** → controls which rows a group can see (e.g., only their region)
- **Column mask** → controls what value they see in a specific column (e.g., real vs. masked email)

These stack independently on the same table in Unity Catalog / Snowflake — Group A might see all US rows with real emails, while Group B sees all rows but with masked emails.

Key point: Masking is enforced at the storage/catalog layer (Unity Catalog, Snowflake, Lake Formation), not in application code — so it's consistent no matter what tool (BI dashboard, notebook, SQL client) is used to query the data. That's what makes it secure and audit-friendly, versus filtering data separately in each application.